PERSONAL EXPENSE MANAGER

Note:

For the post-experiment tasks, I've randomly selected:

1. Add income tracking and calculate total income, expense, and balance.
2. Add search and category filtering to the expense list.

Step 1: Creating the Transaction Model

Transaction_item.dart

```
class TransactionItem {
  final String id;
  final String title;
  final double amount;
  final String category;
  final String type; // 'Expense' or 'Income'
  final DateTime date;

  TransactionItem({
    required this.id,
    required this.title,
    required this.amount,
    required this.category,
    required this.type,
    required this.date,
  });
}
```

Step 2: Main.dart

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Expense Manager',
      theme: ThemeData(primarySwatch: Colors.teal),
      home: const HomeScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}
```

Screens

Step 3: Creating the Home Screen Layout & Dashboard

Code:

```
import 'package:flutter/material.dart';
import '../models/transaction_item.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // Master list of transactions stored in memory
  final List<TransactionItem> _transactions = [];

  // Controllers for the form inputs
  final _titleController = TextEditingController();
  final _amountController = TextEditingController();
  String _selectedCategory = 'Food';
  String _selectedType = 'Expense';

  final List<String> _categories = ['Food', 'Travel', 'College', 'Salary', 'Shopping'];

  @override
```

```
void dispose() {
  _titleController.dispose();
  _amountController.dispose();
  super.dispose();
}

// Calculate Total Income
double get _totalIncome {
  return _transactions
      .where((item) => item.type == 'Income')
      .fold(0.0, (sum, item) => sum + item.amount);
}

// Calculate Total Expense
double get _totalExpense {
  return _transactions
      .where((item) => item.type == 'Expense')
      .fold(0.0, (sum, item) => sum + item.amount);
}

// Calculate Balance
double get _balance => _totalIncome - _totalExpense;

void _addTransaction() {
  if (_titleController.text.isEmpty || _amountController.text.isEmpty) return;

  setState(() {
    _transactions.add(
      TransactionItem(
        id: DateTime.now().toString(),
        title: _titleController.text,
        amount: double.parse(_amountController.text),
        category: _selectedCategory,
        type: _selectedType,
        date: DateTime.now(),
      ),
    );
  });

  _titleController.clear();
  _amountController.clear();
  Navigator.pop(context); // Close bottom sheet
}

void _deleteTransaction(String id) {
  setState(() {
    _transactions.removeWhere((item) => item.id == id);
  });
```

```
  }

  void _showAddModal(BuildContext context) {
    showModalBottomSheet(
      context: context,
      isScrollControlled: true,
      builder: (_) => Padding(
        padding: EdgeInsets.only(
          bottom: MediaQuery.of(context).viewInsets.bottom,
          left: 16,
          right: 16,
          top: 16,
        ),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('Add Transaction', style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
            TextField(controller: _titleController, decoration: const InputDecoration(labelText:
'Description')),
            TextField(controller: _amountController, keyboardType: TextInputType.number,
decoration: const InputDecoration(labelText: 'Amount')),
            DropdownButtonFormField(
              value: _selectedType,
              items: ['Expense', 'Income'].map((val) => DropdownMenuItem(value: val, child:
Text(val))).toList(),
              onChanged: (val) => setState(() => _selectedType = val.toString()),
              decoration: const InputDecoration(labelText: 'Type'),
            ),
            DropdownButtonFormField(
              value: _selectedCategory,
              items: _categories.map((val) => DropdownMenuItem(value: val, child:
Text(val))).toList(),
              onChanged: (val) => setState(() => _selectedCategory = val.toString()),
              decoration: const InputDecoration(labelText: 'Category'),
            ),
            const SizedBox(height: 20),
            ElevatedButton(onPressed: _addTransaction, child: const Text('Save Transaction')),
            const SizedBox(height: 20),
          ],
        ),
      ),
    );
  }

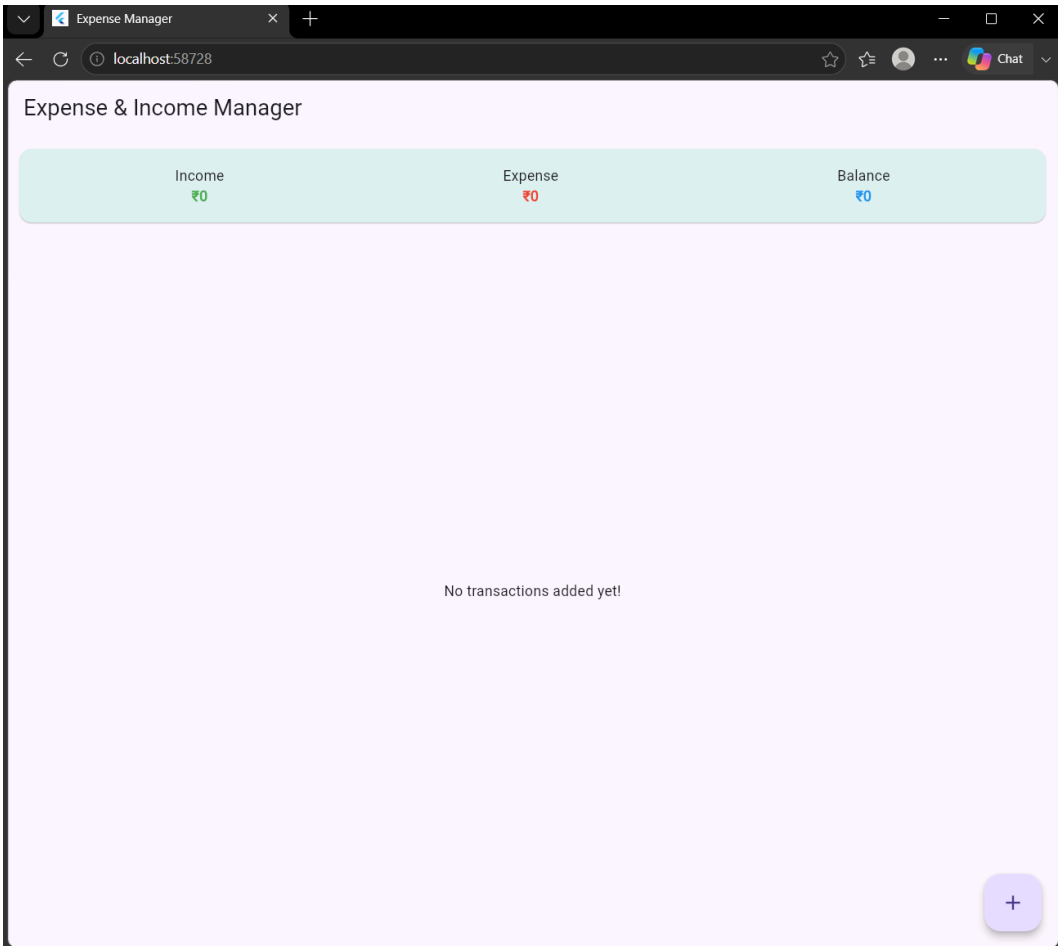
  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
    appBar: AppBar(title: const Text('Expense & Income Manager')),
    body: Column(
      children: [
        // Dashboard Summary Card (Income, Expense, Balance)
        Card(
          margin: const EdgeInsets.all(12),
          color: Colors.teal.shade50,
          child: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.spaceAround,
              children: [
                Column(children: [const Text('Income'), Text('₹$_totalIncome', style: const
TextStyle(color: Colors.green, fontWeight: FontWeight.bold))]),
                Column(children: [const Text('Expense'), Text('₹$_totalExpense', style: const
TextStyle(color: Colors.red, fontWeight: FontWeight.bold))]),
                Column(children: [const Text('Balance'), Text('₹$_balance', style: TextStyle(color:
_balance >= 0 ? Colors.blue : Colors.red, fontWeight: FontWeight.bold))]),
              ],
            ),
          ),
        ),

        // Transaction List View
        Expanded(
          child: _transactions.isEmpty
              ? const Center(child: Text('No transactions added yet!'))
              : ListView.builder(
                  itemCount: _transactions.length,
                  itemBuilder: (ctx, index) {
                    final tx = _transactions[index];
                    return ListTile(
                      leading: CircleAvatar(
                        backgroundColor: tx.type == 'Income' ? Colors.green.shade100 :
Colors.red.shade100,
                        child: Icon(tx.type == 'Income' ? Icons.arrow_downward :
Icons.arrow_upward,
                            color: tx.type == 'Income' ? Colors.green : Colors.red),
                      ),
                      title: Text(tx.title, style: const TextStyle(fontWeight: FontWeight.bold)),
                      subtitle: Text('${tx.category} • tx.date.day/{tx.date.month}/${tx.date.year}'),
                      trailing: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          Text(
                            'tx.type == 'Income'? ' + ': ' − '₹{tx.amount}',
                            style: TextStyle(color: tx.type == 'Income' ? Colors.green : Colors.red,
fontWeight: FontWeight.bold),
```

```
                    ),
                    IconButton(
                      icon: const Icon(Icons.delete, color: Colors.grey),
                      onPressed: () => _deleteTransaction(tx.id),
                    ),
                  ],
                ),
              );
            },
          ),
        ),
      ],
    ),
    floatingActionButton: FloatingActionButton(
      onPressed: () => _showAddModal(context),
      child: const Icon(Icons.add),
    ),
  );
 }
}
```

Output:



Step 4: Adding Search and Category Filtering

Code:

```dart
import 'package:flutter/material.dart';
import '../models/transaction_item.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // Master list storing transactions
  final List<TransactionItem> _transactions = [];

  // Search and filter state variables
  String _searchQuery = '';
  String _filterCategory = 'All';
  final List<String> _filterCategories = [
    'All',
    'Food',
    'Travel',
    'College',
    'Salary',
    'Shopping'
  ];

  // Controllers for the input fields
  final _titleController = TextEditingController();
  final _amountController = TextEditingController();
  String _selectedCategory = 'Food';
  String _selectedType = 'Expense';

  final List<String> _categories = [
    'Food',
    'Travel',
    'College',
    'Salary',
    'Shopping'
  ];
```

```
  @override
  void dispose() {
    _titleController.dispose();
    _amountController.dispose();
    super.dispose();
  }

  // Calculate Total Income
  double get _totalIncome {
    return _transactions
        .where((item) => item.type == 'Income')
        .fold(0.0, (sum, item) => sum + item.amount);
  }

  // Calculate Total Expense
  double get _totalExpense {
    return _transactions
        .where((item) => item.type == 'Expense')
        .fold(0.0, (sum, item) => sum + item.amount);
  }

  // Calculate Net Balance
  double get _balance => _totalIncome - _totalExpense;

  // Filtered transactions getter based on search query and category
  List<TransactionItem> get _filteredTransactions {
    return _transactions.where((tx) {
      bool matchesSearch =
          tx.title.toLowerCase().contains(_searchQuery.toLowerCase());
      bool matchesCategory =
          _filterCategory == 'All' || tx.category == _filterCategory;
      return matchesSearch && matchesCategory;
    }).toList();
  }

  void _addTransaction() {
    if (_titleController.text.isEmpty ||
_amountController.text.isEmpty) return;

    setState(() {
      _transactions.add(
        TransactionItem(
```

```
          id: DateTime.now().toString(),
          title: _titleController.text,
          amount: double.parse(_amountController.text),
          category: _selectedCategory,
          type: _selectedType,
          date: DateTime.now(),
        ),
      );
    });

    _titleController.clear();
    _amountController.clear();
    Navigator.pop(context);
  }

  void _deleteTransaction(String id) {
    setState(() {
      _transactions.removeWhere((item) => item.id == id);
    });
  }

  void _showAddModal(BuildContext context) {
    showModalBottomSheet(
      context: context,
      isScrollControlled: true,
      builder: (_) => Padding(
        padding: EdgeInsets.only(
          bottom: MediaQuery.of(context).viewInsets.bottom,
          left: 16,
          right: 16,
          top: 16,
        ),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            const Text('Add Transaction',
                style: TextStyle(fontSize: 18, fontWeight:
FontWeight.bold)),
            TextField(
                controller: _titleController,
                decoration: const InputDecoration(labelText:
'Description')),
            TextField(
```

```
                controller: _amountController,
                keyboardType: TextInputType.number,
                decoration: const InputDecoration(labelText:
'Amount')),
            DropdownButtonFormField(
              value: _selectedType,
              items: ['Expense', 'Income']
                  .map((val) => DropdownMenuItem(value: val, child:
Text(val)))
                  .toList(),
              onChanged: (val) =>
                  setState(() => _selectedType = val.toString()),
              decoration: const InputDecoration(labelText: 'Type'),
            ),
            DropdownButtonFormField(
              value: _selectedCategory,
              items: _categories
                  .map((val) => DropdownMenuItem(value: val, child:
Text(val)))
                  .toList(),
              onChanged: (val) =>
                  setState(() => _selectedCategory = val.toString()),
              decoration: const InputDecoration(labelText: 'Category'),
            ),
            const SizedBox(height: 20),
            ElevatedButton(
                onPressed: _addTransaction,
                child: const Text('Save Transaction')),
            const SizedBox(height: 20),
          ],
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Expense & Income Manager')),
      body: Column(
        children: [
          // Dashboard Summary Card
          Card(
```

```
            margin: const EdgeInsets.all(12),
            color: Colors.teal.shade50,
            child: Padding(
              padding: const EdgeInsets.all(16.0),
              child: Row(
                mainAxisAlignment: MainAxisAlignment.spaceAround,
                children: [
                  Column(children: [
                    const Text('Income'),
                    Text('₹$_totalIncome',
                        style: const TextStyle(
                            color: Colors.green, fontWeight:
FontWeight.bold))
                  ]),
                  Column(children: [
                    const Text('Expense'),
                    Text('₹$_totalExpense',
                        style: const TextStyle(
                            color: Colors.red, fontWeight:
FontWeight.bold))
                  ]),
                  Column(children: [
                    const Text('Balance'),
                    Text('₹$_balance',
                        style: TextStyle(
                            color: _balance >= 0 ? Colors.blue :
Colors.red,
                            fontWeight: FontWeight.bold))
                  ]),
                ],
              ),
            ),
          ),

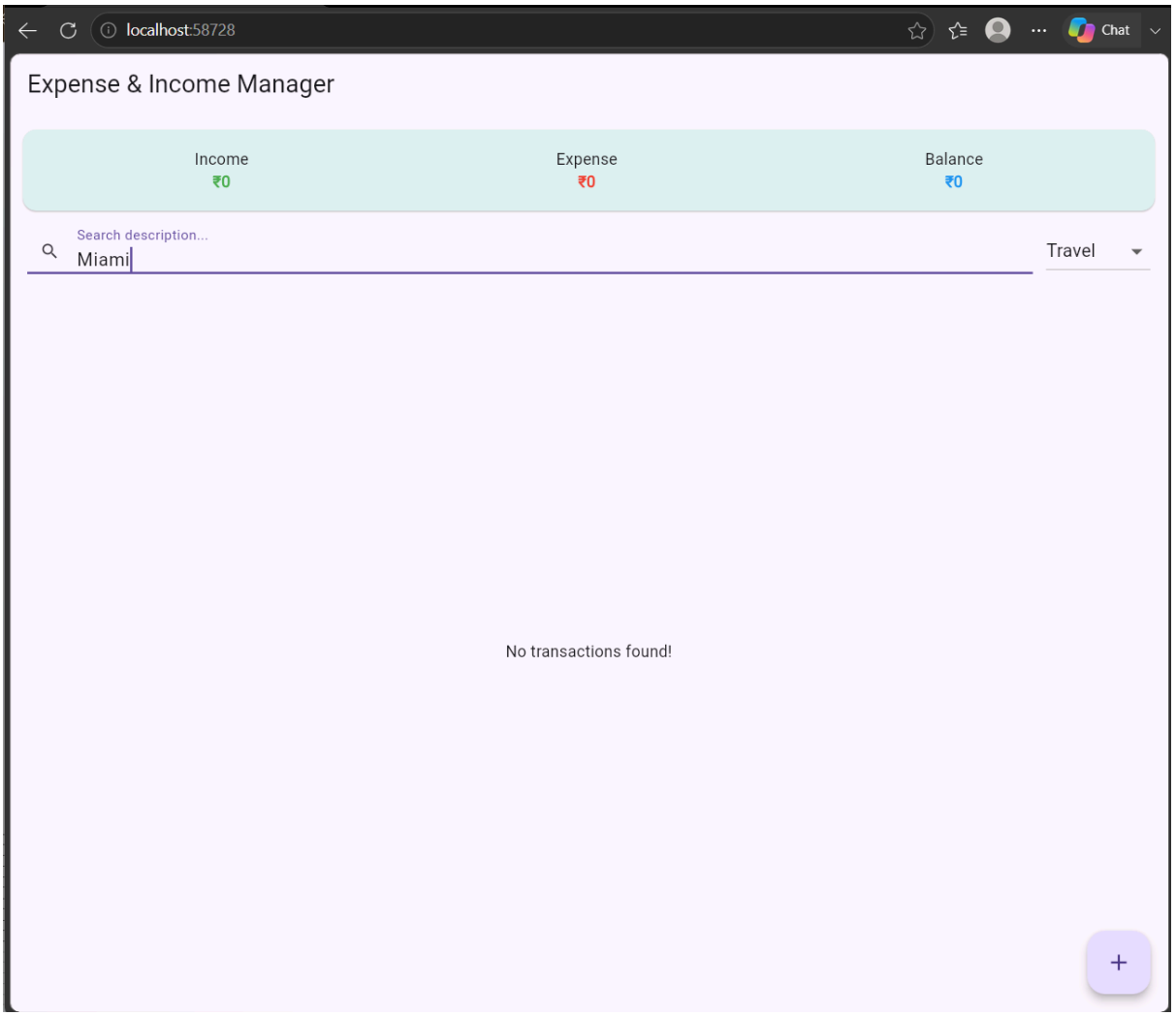
          // Search and Category Filter Bar (Step 4)
          Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16.0),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    decoration: const InputDecoration(
                      labelText: 'Search description...',
```

```
                    prefixIcon: Icon(Icons.search),
                    isDense: true,
                  ),
                  onChanged: (val) => setState(() => _searchQuery =
val),
                ),
              ),
              const SizedBox(width: 12),
              DropdownButton<String>(
                value: _filterCategory,
                items: _filterCategories
                    .map((cat) =>
                        DropdownMenuItem(value: cat, child:
Text(cat)))
                    .toList(),
                onChanged: (val) => setState(() => _filterCategory =
val!),
              ),
            ],
          ),
        ),
        const SizedBox(height: 10),

        // Filtered Transaction List View
        Expanded(
          child: _filteredTransactions.isEmpty
              ? const Center(child: Text('No transactions found!'))
              : ListView.builder(
                  itemCount: _filteredTransactions.length,
                  itemBuilder: (ctx, index) {
                    final tx = _filteredTransactions[index];
                    return ListTile(
                      leading: CircleAvatar(
                        backgroundColor: tx.type == 'Income'
                            ? Colors.green.shade100
                            : Colors.red.shade100,
                        child: Icon(
                            tx.type == 'Income'
                                ? Icons.arrow_downward
                                : Icons.arrow_upward,
                            color: tx.type == 'Income'
                                ? Colors.green
                                : Colors.red),
```

```
                        ),
                        title: Text(tx.title,
                            style:
                                const TextStyle(fontWeight:
FontWeight.bold)),
                        subtitle: Text(
                            '${tx.category} •
${tx.date.day}/${tx.date.month}/${tx.date.year}'),
                        trailing: Row(
                          mainAxisSize: MainAxisSize.min,
                          children: [
                            Text(
                              '${tx.type == 'Income' ? '+' :
'-'}₹${tx.amount}',
                              style: TextStyle(
                                  color: tx.type == 'Income'
                                      ? Colors.green
                                      : Colors.red,
                                  fontWeight: FontWeight.bold),
                            ),
                            IconButton(
                              icon:
                                  const Icon(Icons.delete, color:
Colors.grey),
                              onPressed: () =>
_deleteTransaction(tx.id),
                            ),
                          ],
                        ),
                      );
                    },
                  ),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => _showAddModal(context),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Output:


localhost:58728
Chat
Expense & Income Manager
Income
₹0
Expense
₹0
Balance
₹0
Search description...
Miami
Travel
No transactions found!